

PROGRAMMATION FONCTIONNELLE via le langage Haskell

A. OUALI

L2 au département Mathématique-Informatique
UPR des sciences
Université de Caen Normandie



Système de typage type

- Il est possible de paramétrer la définition du nouveau type

```
type Ensemble a = [a]
```

- Types avec plus d'un paramètre sont également possibles.

```
type Assoc k v = [(k, v)]

find :: Eq k -> k -> Assoc k v -> v
find k t = head [v | (k', v) <- t, k == k']
```

A. Ouali

Programmation fonctionnelle en Haskell

2 / 19

Exercice fonction

- Nous considérons dans cet exercice l'addition et la multiplication sur les matrices. Pour représenter les matrices dans Haskell, nous utilisons une liste de listes d'entiers. Par exemple :

La matrice : $\begin{pmatrix} 1 & 4 & 9 \\ 2 & 5 & 7 \end{pmatrix} \rightarrow \begin{matrix} [1, 4, 9], \\ [2, 5, 7] \end{matrix}$

- Définir un type appelé `Matrice` pour représenter les matrices comme une liste de listes d'entiers. Notre première tâche est d'écrire un test pour vérifier si cette représentation nous donne est une matrice. Ce test doit vérifier deux conditions :
 - que les listes d'entiers sont toutes de longueur égale,
 - qu'il y a au moins une ligne et une colonne pour avoir une matrice.
- Définir une fonction `uniforme :: [Int] -> Bool` qui teste si les entiers d'une liste sont tous égaux. Vous pouvez utiliser la fonction d'ordre supérieur `all`. Proposer une autre définition équivalente en utilisant `foldr` et `map`.
- En utilisant votre fonction `uniforme`, définir une fonction `valide :: Matrix -> Bool` qui teste si une liste de listes d'entiers est une matrice (elle devrait tester les propriétés 1) et 2) spécifiées ci-dessus).

A. Ouali

Programmation fonctionnelle en Haskell

3 / 19

Exercice fonction

- Une autre fonction utile pour travailler avec des paires est `uncurry`, qui transforme une fonction qui prend deux arguments en une fonction qui opère sur une paire.

```
Main> uncurry (+) (10,8)
18
```

- Une fonction d'ordre supérieur utile est `zipWith`. Elle ressemble beaucoup à la fonction `zip` ou `myZip`. La différence est qu'au lieu de combiner des éléments par paire, vous pouvez donner à `zipWith` une fonction spécifique pour combiner chacun des deux éléments.

```
*Main> zipWith (*) [1,2,3] [4,5,6]
[4,10,18]
```

- Définir `myZipWith` en utilisant `zip` et les fonctions d'ordre supérieur `map` et `uncurry`.

A. Ouali

Programmation fonctionnelle en Haskell

4 / 19

Exercice fonction

- L'addition de deux matrices de taille égale se fait en ajoutant par paires les éléments qui sont dans la même position, par exemple :

$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} + \begin{pmatrix} 10 & 20 & 30 \\ 40 & 50 & 60 \end{pmatrix} = \begin{pmatrix} 11 & 22 & 33 \\ 44 & 55 & 66 \end{pmatrix}$

- Définir une fonction `plusM` qui additionne deux matrices. Renvoie une erreur si l'entrée ne convient pas. Il peut être utile de définir d'autres fonctions pour définir la fonction.

A. Ouali

Programmation fonctionnelle en Haskell

5 / 19

Système de typage data : déclaration

- Introduction d'un type **complètement nouveau** par opposition à un type existant dans structure
- Haskell propose un mécanisme **data** pour **regrouper les valeurs** du nouveau type
- Par exemple :

```
data Boolean = Vrai | Faux
```

- Le symbole `|` est interprété comme "ou"
- Les nouvelles valeurs (`Vrai` et `Faux`) sont appelées des constructeurs du nouveau type `Boolean`
- Le **nom** du nouveau type ainsi que les constructeurs **doivent commencer par une lettre en majuscule**
- un **constructeur** ne peut être utilisé qu'une **seule fois**
- Aucun héritage du système utilisé :

```
data Montype = Valeur1 | Valeur2 | Valeur3
-- ...
```

A. Ouali

Programmation fonctionnelle en Haskell

6 / 19

Système de typage data : utilisation

Les nouvelles valeurs (constructeurs) sont utilisées comme les valeurs primitives :

- passées en **arguments** aux fonctions
- renvoyées** en tant que résultats de fonctions
- stockées** dans des **structures** de données telles que des listes ou autres

```
data Deplacement = Haut | Bas | Gauche | Droite
type Position = (Int,Int) -- des positions sur un plan de deux dimensions
```

- Fonctions qui appliquent un mouvement à une position :

```
-- applique un déplacement à une position courante
deplace :: Deplacement -> Position -> Position
deplace Gauche (x, y) = (x - 1, y)
deplace Droite (x, y) = (x + 1, y)
deplace Haut (x, y) = (x, y + 1)
deplace Bas (x, y) = (x, y - 1)
-- applique une série de déplacements
deplaces :: [Deplacement] -> Position -> Position
deplaces [] p = p
deplaces (m:ms) p = deplaces ms (deplace m p)
-- applique un déplacement inverse
flip :: Deplacement -> Deplacement
flip Gauche = Droite
flip Droite = Gauche
flip Haut = Bas
flip Bas = Haut
```

A. Ouali

Programmation fonctionnelle en Haskell

7 / 19

Système de typage data : constructeur avec des arguments

- Les constructeurs d'une déclaration de données peuvent également avoir des arguments
- Type de `Forme` qui comprend des cercles avec un rayon et des rectangles avec des dimensions

```
data Forme = Cercle Float | Rectangle Float Float
```

- Le type `Forme` a par exemple comme valeur `Cercle 1.12`
- Ces constructeurs sont utilisés pour définir des fonctions sur des formes

```
-- retourne un carré qui est un rectangle avec x = y
carré :: Float -> Forme
carré n = Rectangle n n
-- calcule la surface d'une Forme
surface :: Forme -> Float
surface (Cercle r) = pi * r ^ 2
surface (Rectangle x y) = x * y
```

- Les constructeurs `Cercle` et `Rectangle` sont en fait des **fonctions constructeurs** mais sans définition

```
*Main> :t Cercle
Cercle :: Float -> Forme
```

A. Ouali

Programmation fonctionnelle en Haskell

8 / 19

Système de typage data : données paramétrées



- Similaire au mécanisme **type** pour les types paramétrés
- La bibliothèque standard déclare le type suivant :

```
data Maybe varType = Nothing | Just varType
```

- Une valeur de type `Maybe varType` est soit `Nothing`, soit de la forme `Just x` pour une valeur `x` de type `varType`
- utile quand par exemple les valeurs de type `varType` peuvent échouer :

```
-- faire une division entière plus sûre
safediv :: Int -> Int -> Maybe Int
safediv _ 0 = Nothing
safediv m n = Just (m `div` n)

-- récupérer de façon plus sûre l'élément dans l'entête de la liste
safehead :: [a] -> Maybe a
safehead [] = Nothing
safehead xs = Just (head xs)
```

Système de typage data : types récursifs (1)



- Les types déclarés avec **data** peuvent également être récursifs
 ➔ ce n'est pas le cas du mécanisme **type**
- Un type **Nat** qui a une valeur `Zero`, ou une valeur de la forme `Succ n` pour une valeur `n` de type `Nat`

```
data Nat = Zero | Succ Nat
```

- Une déclaration donne lieu à une séquence **infinie de valeurs** en commençant par la valeur `Zero`

```
Zero
Succ Zero
Succ (Succ Zero)
Succ (Succ (Succ Zero))
...
```

- Les valeurs de type `Nat` peuvent être considérées comme des nombres naturels (`Succ` → 1+)

```
-- conversion Nat vers Int
nat2int :: Nat -> Int
nat2int Zero = 0
nat2int (Succ n) = 1 + nat2int n

-- conversion Int vers Nat
int2nat :: Int -> Nat
int2nat 0 = Zero
int2nat n = Succ (int2nat (n-1))
```

Système de typage data : types récursifs (2)



- On peut déclarer notre propre version du type intégré de **listes** :

```
data Liste varType = Vide | Element varType (Liste varType) deriving Show
```

- une valeur de type `Liste varType` est soit une :

- `Vide` représentant une **liste vide**
- `Cons x xs` représentant une **liste non-vide** pour certaines valeurs `x :: varType` et `xs :: Liste varType`

- nous pouvons définir nos propres versions de fonctions de bibliothèque sur des listes :

```
-- une fonction qui calcule la longueur de notre liste Liste
len :: Liste a -> Int
len Vide = 0
len (Element _ xs) = 1 + len xs

-- évaluation dans l'interpréteur
Prelude> let xs = Element 1 (Element 2 (Element 3 Vide))
Prelude> xs
Element 1 (Element 2 (Element 3 Vide))
Prelude> len xs
3
```

Déclaration des classes et instances



- une nouvelle classe peut être déclarée en utilisant le système **classe** d'Haskell
- La classe `Eq` est déclarée dans la bibliothèque standard comme suit :

```
class MyEq a where
  (==), (/=) :: a -> a -> Bool
  -- une définition par défaut de la différence
  -- en fonction de l'égalité qui sera prochainement définie
  x /= y = not (x == y)
```

- déclarer une instance de cette classe ne nécessite qu'une définition de `==` pour le type concerné :

```
instance MyEq Bool where
  False == False = True
  True == True = True
  _ == _ = False
```

```
*Main> True == True
True
*Main> True /= True
False
*Main> True /= True
False
```

- n'est possible qu'avec le système de typage **data**

Système de typage data : structure de données



- Pour déclarer une structure de données composée de plusieurs champs en Haskell, on utilise la syntaxe suivante

```
data Personne = Personne { nom::String, age::Int} deriving Show
```

- Chaque nom d'un champ pourrait être utilisé comme un fonction pour récupérer la donnée enregistrée dans le champ

```
*Main> age (Personne "Pierre" 45)
45
*Main> nom (Personne "Pierre" 45)
"Pierre"
*Main> !t nom
nom :: Personne -> String
*Main> !t age
age :: Personne -> Int
```

Système de typage data : types récursifs (2)



- On peut faire des opérations en faisant des conversions vers `Int`

```
-- addition entre deux valeur Nat
addition :: Nat -> Nat -> Nat
addition m n = int2nat (nat2int m + nat2int n)

-- évaluation dans l'interpréteur
*Main> addition (Succ (Succ (Succ Zero))) (Succ (Succ (Succ Zero)))
Succ (Succ (Succ (Succ (Succ Zero))))

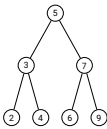
*Main> nat2int (Succ (Succ (Succ (Succ (Succ (Succ Zero))))))
6

-- redéfinir sans avoir besoin de telles conversions en utilisant le pattern-matching
add Zero n = n
add (Succ m) n = Succ (add m n)
```

Système de typage data : exercice



- Une des structures de données les plus utilisées est un arbres binaire :



- les chiffres 2, 4, 6, 9 apparaissent sur les feuilles de l'arbre
- les chiffres 5, 3, 7 apparaissent sur les nœuds internes

- En utilisant la **récursivité**, définir un type approprié pour représenter de tels arbres.

```
arbre = (Noeud (Noeud (Feuille 2) 3 (Feuille 4)) 5 (Noeud (Feuille 6) 7 (Feuille 9)))
```

- Définir une fonction qui décide si un entier donné apparaît dans un arbre :

```
apparaît :: Int -> Arbre -> Bool
```

- Définir une fonction qui aplatit un arbre en une liste :

```
aplatir :: Arbre -> [Int]
```

Déclaration des classes et instances

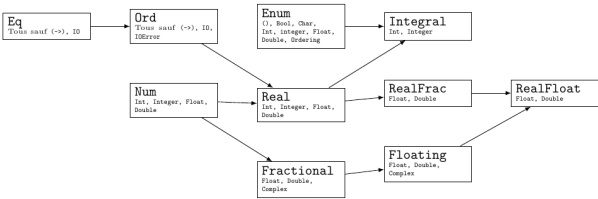


- Revenons au type `Nat` que nous avons défini précédemment :

```
data Nat = Zero | Succ Nat
```

- Créer une instance pour associer le type `Nat` à la classe `MyEq`.

```
*Main> (Succ (Succ (Succ Zero))) == (Succ (Succ (Succ Zero)))
True
*Main> (Succ (Succ (Succ Zero))) == (Succ (Succ Zero))
False
```



► Système de dérivation : toutes les fonctions membres des classes dérivées sont utilisées avec des valeurs du nouveau type

```
data Boolean = Vrai | Faux
deriving (Eq, Ord, Show, Read)

*Main> Faux == Faux
True
*Main> Faux < Vrai
False
*Main> show Faux
"Faux"
*Main> read "Faux"::Boolean
Faux
```

- Un arbre binaire est dit équilibré si le nombre de feuilles dans les sous-arbres gauche et droit de chaque nœud diffère d'au plus un, sachant que les feuilles sont trivialement équilibrées. Définir une fonction `balanced :: Arbre -> Bool` qui décide si un arbre est équilibré ou non. Indication : définir une fonction qui renvoie le nombre de feuilles dans un arbre.
- En utilisant le type `Arbre`, définissez une fonction `balance :: [Int] -> Arbre` qui convertit une liste non vide d'entiers en un arbre équilibré.
- Définir une fonction qui compare si deux arbres de type `Arbre` sont identiques avec l'opérateur `==`.